

华中科技大学

深度学习

实验一 多层神经网络的 numpy 实现

时间： 2026. 03. 18
班级： 电信 2507
姓名： 王兆鹏
学号： M202573183

目录

一、实验题目与要求	3
二、实验目的	3
三、模块实现与流程图	3
3.1 网络基础框架	3
3.2 损失函数与 L2 正则化	5
3.3 Dropout 层	5
3.4 Xavier 与 He 初始化	6
3.5 实验框图	7
四、实验结果与讨论	7
4.1 不同初始化方式的实验结果	7
4.2 参数调优实验结果	8
4.3 混淆矩阵与分类错误案例可视化	10
4.4 超参数敏感性分析	12
4.5 实验分析与讨论	13

一、实验题目与要求

本实验题目为“多层神经网络的 NumPy 实现”。实验要求仅使用 NumPy 手动完成面向 MNIST 手写数字分类的多层全连接神经网络设计与训练，实现前向传播、反向传播、激活函数、交叉熵损失、L2 正则化等，通过超参数对比分析使测试集准确率达到 95%以上。

二、实验目的

1. 掌握全连接神经网络的前向传播与反向传播原理
2. 理解激活函数、损失函数、正则化方法的作用
3. 通过手动实现神经网络，深入理解深度学习框架底层机制
4. 探索超参数对模型性能的影响规律

三、模块实现与流程图

3.1 网络基础框架

3.1.1 全连接层实现

```
class FullyConnectedLayer():
    def __init__(self, input_dim, output_dim, activation=None, d_activation=None,
method="norm"):
        scale = init_stg(method = method, input_dim = input_dim)
        self.W = (np.random.randn(input_dim, output_dim) * scale).astype(np.float32)
        self.b = np.zeros((1, output_dim), dtype=np.float32)
        self.activation = activation
        self.d_activation = d_activation

    def forward(self, X):
        self.X = X
        self.Z = X @ self.W + self.b
        if self.activation is None:
            self.A = self.Z
        else:
            self.A = self.activation(self.Z)
        return self.A

    def backward(self, dA):
        batch_size = self.X.shape[0]
        if self.activation == None:
            dZ = dA
        else :
```

```

        dZ = dA * self.d_activation(self.Z)
    self.dW = (self.X.T @ dZ) / batch_size
    self.db = np.sum(dZ, axis=0, keepdims=True) / batch_size
    dX = dZ @ self.W.T
    return dX

def update(self, lr):
    self.W = self.W - lr * self.dW
    self.b = self.b - lr * self.db

```

3.1.2 MNIST 识别网络实现

```

class Net():
    def __init__(self, hidden_num1=256, hidden_num2=128, activation=relu,
d_activation=d_relu, lr=0.01, method="norm", dropout_rate=0.0):
        self.fc1 = FullyConnectedLayer(784, hidden_num1, activation=activation,
d_activation=d_activation, method=method)
        self.fc2 = FullyConnectedLayer(hidden_num1, hidden_num2, activation=activation,
d_activation=d_activation, method=method)
        self.fc3 = FullyConnectedLayer(hidden_num2, 10, activation=None, d_activation=None,
method=method)

        self.layers = [self.fc1, self.fc2, self.fc3]
        self.lr = lr
        eps = 1e-12
        if dropout_rate > eps:
            print("Dropout rate:", dropout_rate)
            self.drop1 = DropOutLayer(dropout_rate)
            self.drop2 = DropOutLayer(dropout_rate)
            self.layers = [self.fc1, self.drop1, self.fc2, self.drop2, self.fc3]

    def forward(self, X, training = True):
        for layer in self.layers:
            if isinstance(layer, DropOutLayer):
                X = layer.forward(X, training=training)
            else:
                X = layer.forward(X)
        y_pred = softmax(X)
        y_pred_label = np.argmax(y_pred, axis=1)
        return y_pred, y_pred_label

    def backward(self, y_true, y_pred):
        dA = y_pred - y_true
        for layer in reversed(self.layers):
            dA = layer.backward(dA)

```

```
def update(self):
    for layer in self.layers:
        if isinstance(layer, FullyConnectedLayer):
            layer.update(self.lr)
```

3.2 损失函数与 L2 正则化

损失函数包括交叉熵损失函数和 L2 正则化部分，其中，交叉熵损失的公式是：

$$L_{CE} = - \sum_{i=1}^n y_i \log \left(\frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \right)$$

L2 正则化主要用于防止模型过拟合，直观上理解就是 L2 正则化是对于大数值的权重向量进行严厉惩罚，其公式是：

$$L = L(W) + \lambda \sum_{i=1}^n w_i^2$$

实现的代码如下：

```
def ce_loss(self, y_true, y_pred):
    y_pred = np.clip(y_pred, self.eps, 1.0)
    return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))

def l2_loss(self, model):
    if self.l2_lambda <= 0:
        return 0.0

    reg = 0.0
    for layer in model.layers:
        if isinstance(layer, FullyConnectedLayer):
            reg += np.sum(layer.W ** 2)

    return 0.5 * self.l2_lambda * reg
```

3.3 Dropout 层

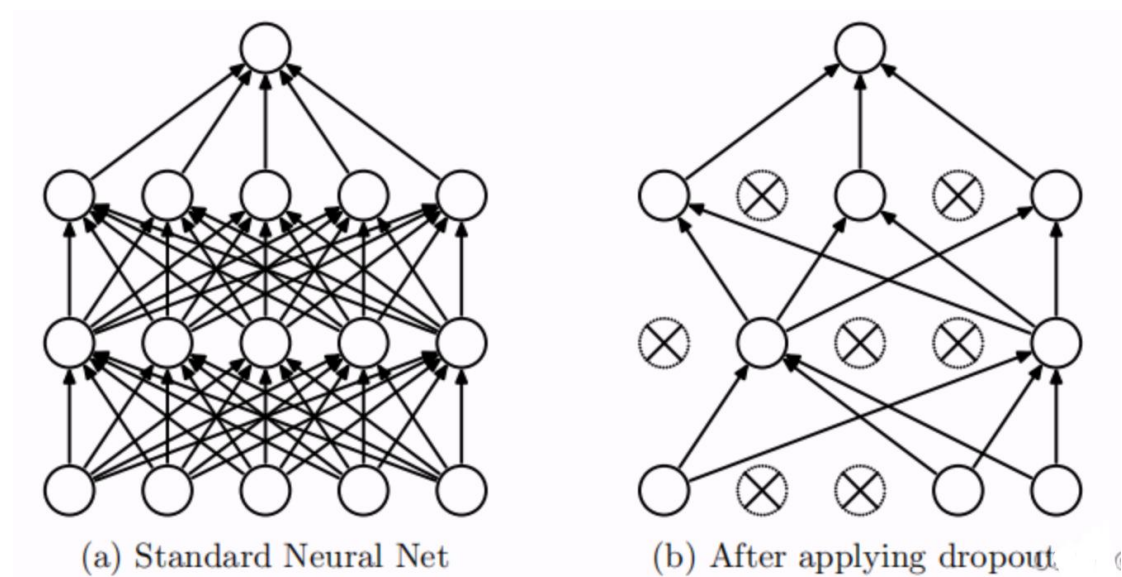


图 3-1 Dropout 层结构示意

实现的代码如下：

```
class DropOutLayer():
    def __init__(self, dropout_rate):
        self.dropout_rate = dropout_rate
        self.mask = None

    def forward(self, X, training = True):
        if (not training):
            return X * (1.0 - self.dropout_rate)
        else:
            self.mask = (np.random.rand(*X.shape) > self.dropout_rate).astype(X.dtype)
            return X * self.mask

    def backward(self, dA):
        return dA * self.mask
```

3.4 Xavier 与 He 初始化

Xavier 初始化的公式是：

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

其更适合 sigmoid, tanh 等激活函数。

He 的公式是：

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

其更适合 ReLU, LeakyReLU 等激活函数。

实现的代码如下：

```
def init_stg(method = "norm", input_dim = 256, output_dim = 512):  
    if method == "norm":  
        scale = 0.01  
    if method == "Xavier":  
        scale = math.sqrt(2.0 / input_dim + output_dim)  
    if method == "He":  
        scale = math.sqrt(2.0 / input_dim)  
    return scale
```

3.5 实验框图

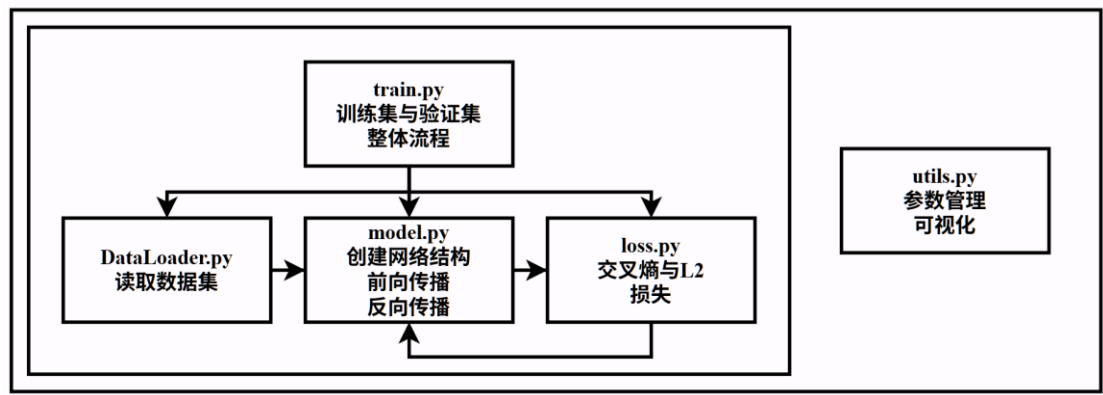


图 3-2 实验模块功能与流程框图

四、实验结果与讨论

4.1 不同初始化方式的实验结果

选择不同的初始化方式，隐藏层层数为 2，神经元个数分别为 256 和 128，使用 ReLU 激活函数，epoch=10，得到了如下实验结果：

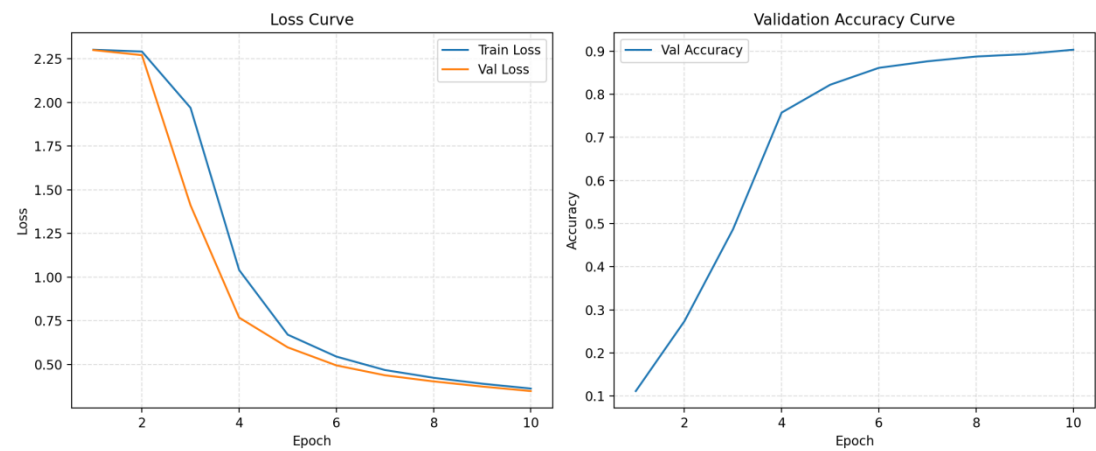


图 4-1 文档所给初始化方式训练 10 轮的损失与验证集准确率

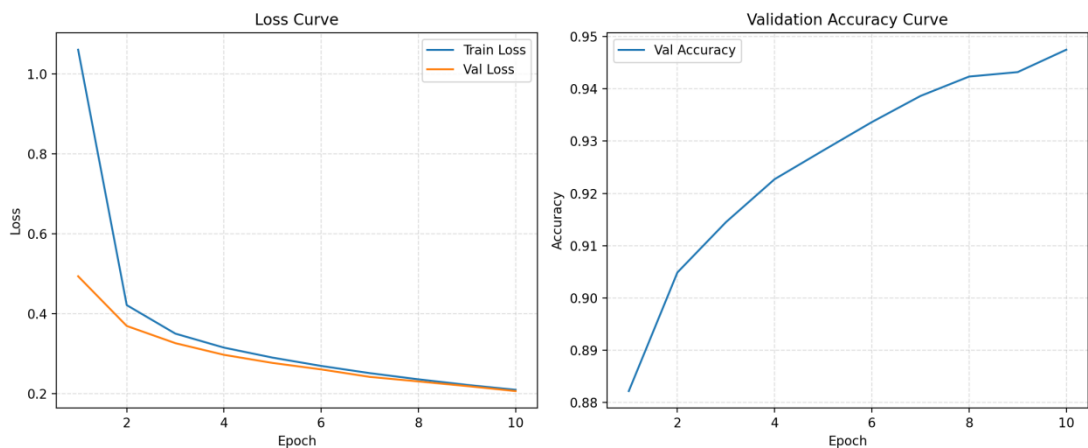


图 4-2 Xavier 初始化方式训练 10 轮的损失与验证集准确率

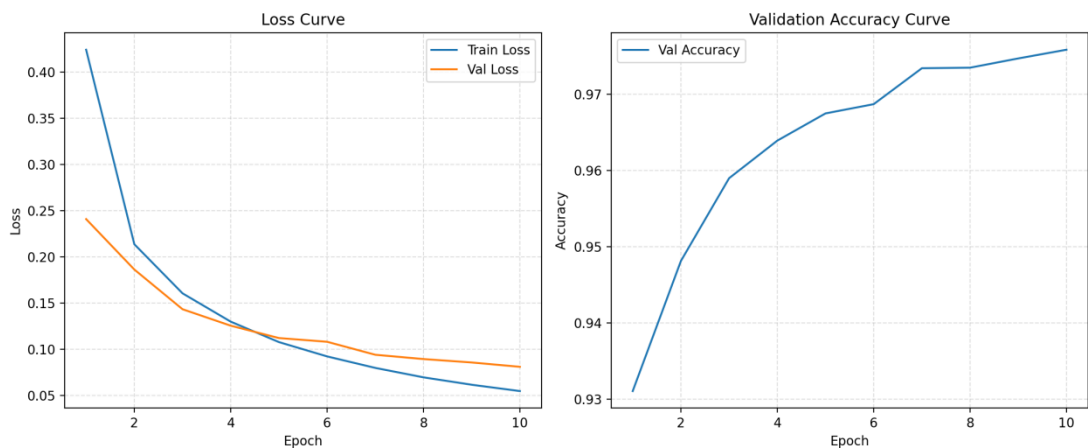


图 4-3 He 初始化方式训练 10 轮的损失与验证集准确率

4.1.2 初始化方式实验分析

在相同网络结构和 ReLU 激活函数条件下，不同初始化方式对训练效果影响显著。文档所给初始化方式收敛较慢，验证集准确率仅约 0.90；Xavier 初始化明显改善了收敛速度与稳定性，准确率提升至约 0.95；He 初始化效果最佳，损失下降最快且最终验证准确率最高，达到约 0.98，更适合 ReLU 网络。后续实验均选用 He 初始化方式。

4.2 参数调优实验结果

表 1 参数调优实验表格

实验组	隐藏层 结构	激活函数	学习率	Batch Size	L2 系数	验证集 准确率 (最佳)

Baseline	[256,128]	ReLU	0.01	64	1e-4	0.9805
组 1	[512,256]	LeakyReLU	0.005	128	1e-3	0.9715
组 2	[128]	Tanh	0.02	32	0	0.9793

4.2.1 各组实验结果的损失曲线与验证集准确率曲线

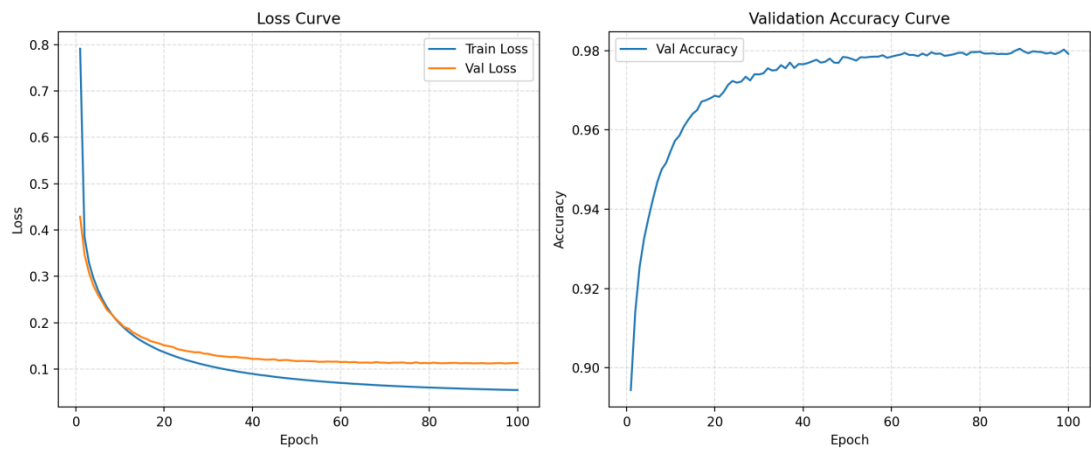


图 4-4 Baseline 组的损失曲线与验证集准确率曲线

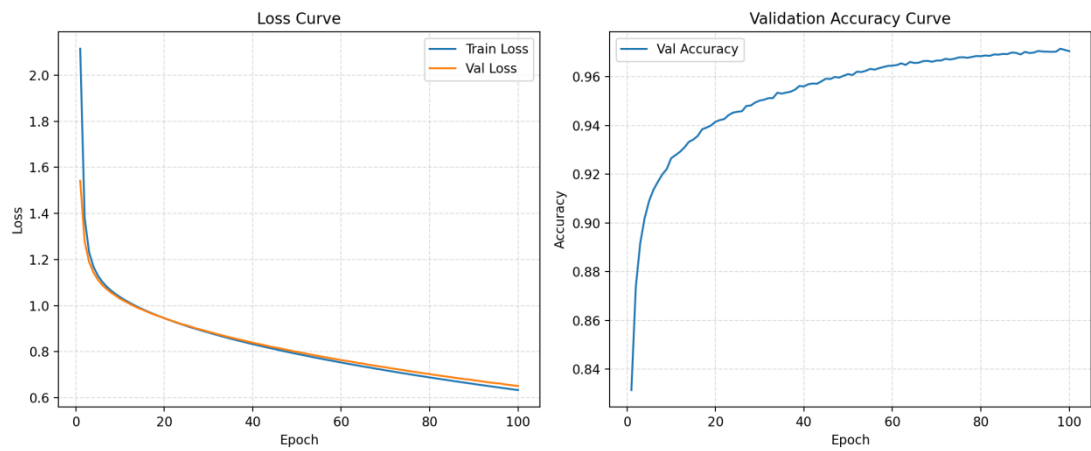


图 4-5 组 1 的损失曲线与验证集准确率曲线

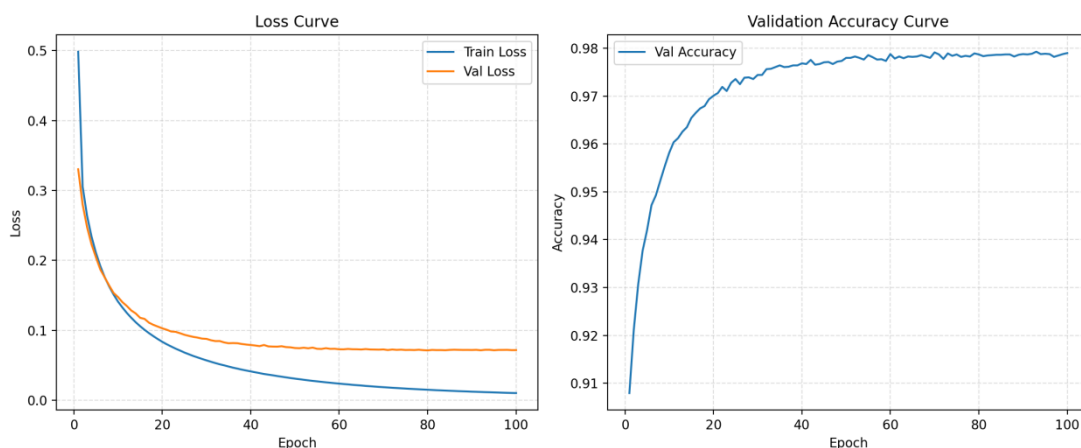


图 4-6 组 2 的损失曲线与验证集准确率曲线

从参数调优结果看, **Baseline** 组综合表现最好, 验证集最佳准确率达到 0.9805, 说明在当前任务中, 该组组合较为合理。组 2 的准确率为 0.9793, 与 **Baseline** 接近, 表明较浅的单隐藏层结构配合 **Tanh**、较大学习率和无正则化也能取得较好效果, 但后期训练损失持续下降而验证损失下降趋缓, 存在一定过拟合倾向。组 1 的准确率最低, 仅为 0.9715, 且损失下降明显慢于另外两组, 说明更深更宽的网络、较小学习率、更大 **Batch Size** 与较强 **L2** 正则的组合未能发挥优势, 可能出现了学习不足和收敛偏慢的问题。综上所述可以发现网络结构与超参数并非越大越强, 需要合理匹配参数以提升性能。

4.3 混淆矩阵与分类错误案例可视化

根据上述实验结果, 取 **baseline** 组作为分析对象, 进行进一步分析。

4.3.1 混淆矩阵

混淆矩阵如图 4-7 所示。可以发现, **Baseline** 组在 MNIST 上整体分类效果较好, 主对角线数值明显高于非对角线, 说明大多数数字都能被正确识别。其中数字 1、4、7、9 的识别表现较稳定, 而 3 与 5、4 与 9、2 与 7 之间存在一定混淆, 说明这些数字在部分书写样本中形态较为相似, 容易造成误判, 但总体误分类数量较少, 模型具有较好的分类能力。

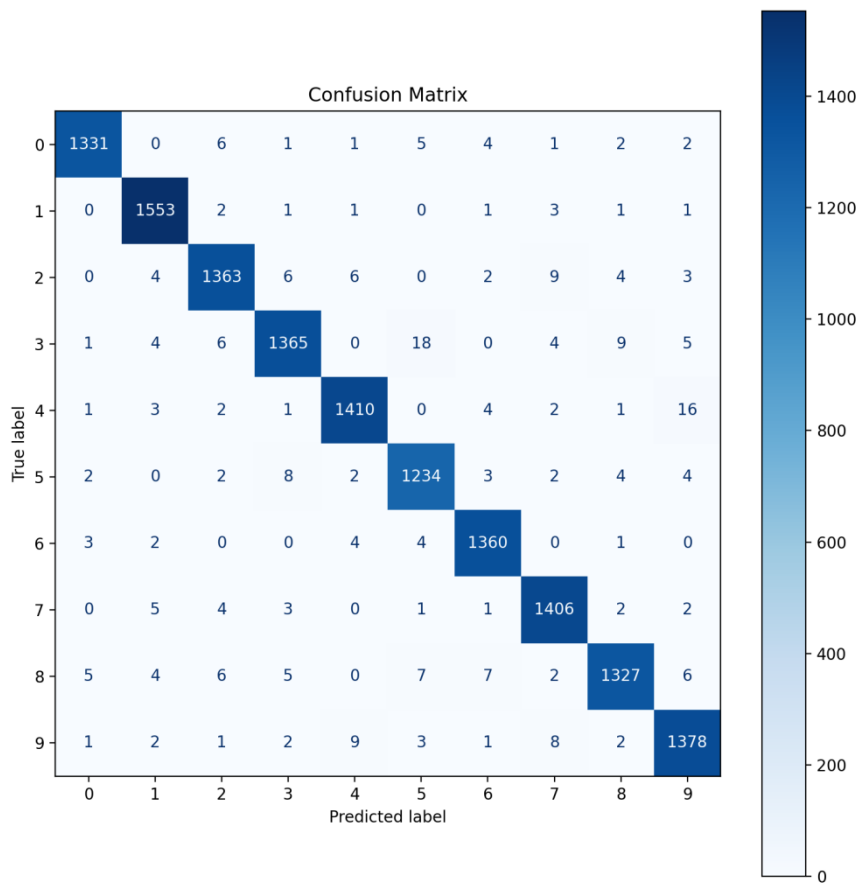


图 4-7 Baseline 组实验结果的混淆矩阵

4.3.2 分类错误案例可视化

分类错误案例的部分展示如图 4-8 所示。结合混淆矩阵与错误样本可视化可以看出，Baseline 组的误分类主要集中在字形相近、书写潦草或局部笔画缺失的样本上。例如 3 易被分成 5、8、0；4 易被分成 9；2 有时会被分成 4 或 7；8 也会被分成 3、9、1。可视化中的错误样本普遍存在倾斜、断笔、粘连、闭环不完整等现象，说明模型虽然整体识别准确率较高，但对边界模糊和个体书写差异较大的样本仍较敏感。

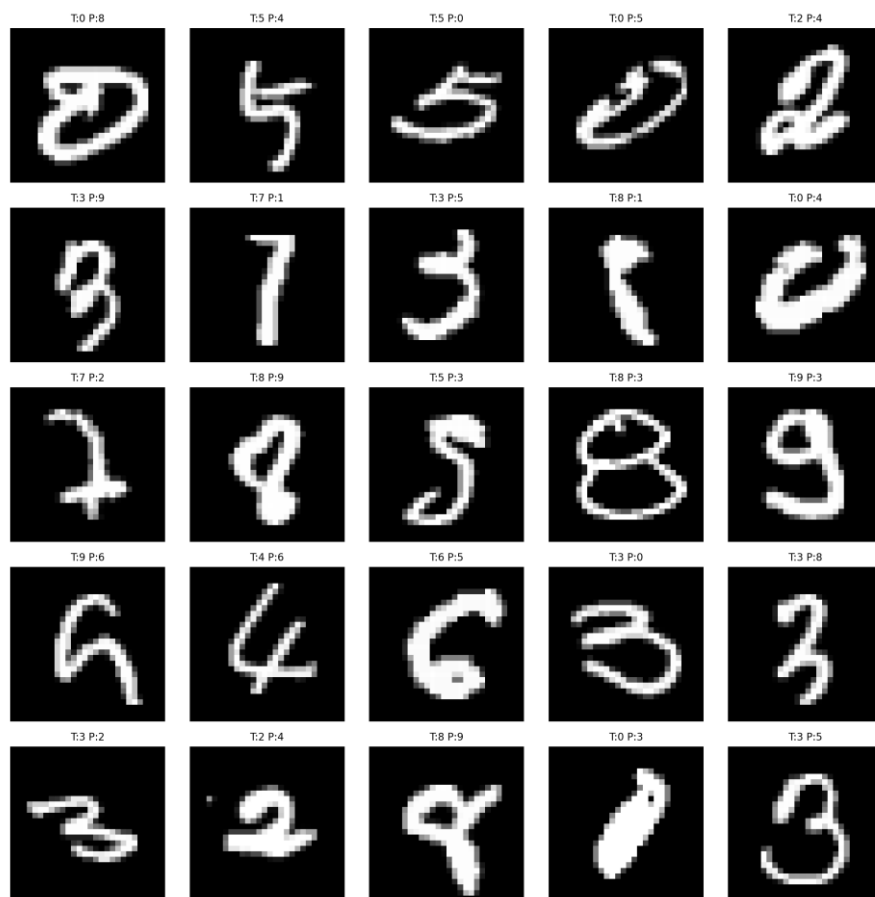


图 4-8 Baseline 组实验结果的分类错误案例可视化分析

4.4 超参数敏感性分析

• 隐藏层结构

隐藏层结构直接决定模型的表达能力与参数规模。本实验中，Baseline 的两层隐藏层结构[256,128]综合表现最好，说明适中的网络宽度与深度能够较好地平衡特征提取能力和训练稳定性。组 1 将网络扩大为[512,256]后，准确率反而下降，表明参数量增大并未带来收益，且更容易造成训练缓慢；组 2 采用单隐藏层[128]虽仍取得较高精度，但整体表征能力略弱于 Baseline。

• 激活函数

激活函数影响网络的非线性表达能力和梯度传播特性。实验结果表明，ReLU 在当前 MNIST 分类任务中表现最稳定，能够较快收敛并获得最高准确率。LeakyReLU 虽能缓解神经元“死亡”问题，但在本实验参数配置下未体现明显优势。Tanh 也取得了较高精度，但其输出容易在饱和区间产生较小梯度，训练后期提升有限。因此，ReLU 更适合作为本实验的默认激活函数。

• Dropout

Dropout 通过在训练阶段随机屏蔽部分神经元，降低特征间的共适应关系，从而增强模型泛化能力。对于 MNIST 这类相对简单的数据集，适度的 Dropout 有助于抑制过拟合，但过大的丢弃率会破坏有效特征表达，导致收敛变慢甚至精度下降。结合本实验情况，若验证集损失与训练集损失差距逐渐扩大，可适当引入小比例 Dropout。

• 学习率

学习率决定参数更新步长，是影响收敛速度和训练稳定性的关键因素。实验中，Baseline 采用学习率 0.01 时取得最佳结果，说明该取值能够在保证稳定训练的同时加快收敛。组 1 学习率为 0.005，损失下降较慢，表现出一定的学习不足；组 2 采用 0.02 虽前期收敛较快，但也更容易引起波动。

• L2 正则化

L2 正则化通过约束权重幅值来降低模型复杂度，从而改善泛化性能。本实验中，Baseline 采用 $1e-4$ 的 L2 系数时效果最好，说明适度正则化有助于提升验证集表现。组 1 的 L2 系数增大到 $1e-3$ 后，模型准确率下降，表明正则化过强会抑制参数学习，使模型出现欠拟合。组 2 不使用 L2 时虽然精度较高，但训练后期训练损失与验证损失差距有所扩大，显示出一定过拟合倾向。

• BatchSize

BatchSize 会影响梯度估计的稳定性以及参数更新频率。实验结果表明，BatchSize 为 64 时综合性能最佳，既能保证较平稳的梯度方向，又能维持较高的更新效率。组 1 使用 128 时，梯度更平滑但更新次数减少，导致收敛偏慢；组 2 使用 32 时，参数更新更加频繁，前期收敛较快，但梯度波动相对更大。

4.5 实验分析与讨论

4.5.1 不同激活函数对梯度消失的影响

不同激活函数对梯度传播能力具有显著影响。Sigmoid 和 Tanh 在输入绝对值较大时容易进入饱和区，其导数趋近于 0，导致反向传播过程中梯度逐层衰减，即出现梯度消失现象。相比之下，ReLU 在正区间内导数恒为 1，能够较好地保持梯度幅值，因此在深层网络中通常具有更快的收敛速度和更好的训练效果。

4.5.2 正则化方法如何平衡偏差-方差

正则化方法的核心作用在于平衡模型的偏差与方差。当模型过于复杂时，虽

然训练误差较低，但容易对训练样本中的噪声和偶然特征产生过度拟合，表现为低偏差、高方差；而适当引入 L2 正则化或 Dropout 后，可以限制模型自由度，使参数分布更加平滑，从而降低方差、提升泛化性能。但如果正则化过强，又会削弱模型对数据规律的拟合能力，导致偏差增大、出现欠拟合。

4.5.3 网络深度与模型性能的关系

网络深度与模型性能并非简单的线性关系。一般而言，增加网络层数有助于模型学习更高层次、更抽象的特征表示，从而提升表达能力；但与此同时，网络越深，参数数量越多，训练难度也越大，对初始化方式、激活函数和优化参数的敏感性更强。若数据集规模有限或任务本身较简单，过深网络不仅难以充分发挥优势，还可能带来收敛缓慢、过拟合甚至性能下降的问题。